

# ARM Linux Kernel

## Interrupt & Driver Internals

A Complete Technical Reference

Author: Sandeep Kumar

Topics: ARM GIC • IRQ Domains • Exception Vectors • SPI Interrupt Flow • Threaded IRQs  
I2C Driver Flow (STK3310) • Tasklets • Workqueues • Softirq vs Tasklet • IRQ Stacks • Interview  
Summary

---

## 1. ARM Interrupt Handling — Why, Where, and How

---

### 1.1 Why Interrupt Handling Exists

Interrupts allow the CPU to respond to asynchronous hardware events (UART RX, DMA completion, GPIO edge, timer tick, etc.) without polling. On ARM SoCs, dozens to hundreds of peripherals need to signal the CPU. The interrupt subsystem provides:

- Efficiency — CPU runs normal code; hardware signals when attention is needed
- Latency control — critical events (timers, network RX) get fast response
- Priority management — higher-priority events preempt lower-priority ones
- Multicore distribution — on SMP systems, interrupts can be routed to specific CPUs

### 1.2 ARM Exception Model

#### ARMv7 (ARM32) Exception Modes

ARM32 has 7 processor modes. Interrupts use two of them:

| Mode | Purpose                                          |
|------|--------------------------------------------------|
| IRQ  | Normal interrupts (nFIQ line low)                |
| FIQ  | Fast interrupts (nFIQ line, dedicated registers) |
| SVC  | Supervisor (kernel normal execution)             |
| ABT  | Abort (memory faults)                            |
| UND  | Undefined instruction                            |
| SYS  | System (privileged, same registers as USR)       |
| USR  | User mode                                        |

Each mode has banked registers (its own sp, lr, and for FIQ: r8–r12). On IRQ entry, hardware automatically: saves CPSR → SPIR\_irq; saves PC+4 → LR\_irq; switches to IRQ mode; disables IRQs; jumps to the IRQ vector.

### ARMv8 (ARM64) Exception Levels

ARM64 replaces modes with Exception Levels (EL):

| EL  | Purpose                    |
|-----|----------------------------|
| EL0 | User space                 |
| EL1 | Kernel (OS)                |
| EL2 | Hypervisor                 |
| EL3 | Secure Monitor (TrustZone) |

Interrupts (IRQ/FIQ) are taken at EL1 when the kernel is running. Hardware automatically: saves PSTATE → SPSR\_EL1; saves PC → ELR\_EL1; switches SP to SP\_EL1; jumps to the vector table base (VBAR\_EL1) + offset.

## 1.3 Exception Vectors

### ARMv7 Vector Table

Fixed at address 0x00000000 (or 0xFFFF0000 with HIVECS, controlled by SCTLR.V):

| Offset | Exception                |
|--------|--------------------------|
| 0x00   | Reset                    |
| 0x04   | Undefined Instruction    |
| 0x08   | SVC (Software Interrupt) |
| 0x0C   | Prefetch Abort           |
| 0x10   | Data Abort               |
| 0x14   | Reserved                 |
| 0x18   | IRQ                      |
| 0x1C   | FIQ                      |

Each entry is a single 32-bit instruction (usually a branch). In Linux ARM32 (arch/arm/kernel/entry-armv.S), the IRQ vector jumps to \_\_irq\_svc (if in SVC mode) or \_\_irq\_usr (if in USR mode).

### ARMv8 Vector Table (VBAR\_EL1)

ARM64 has a 512-byte aligned vector table with 16 entries (4 exception types × 4 sources). Each 128-byte slot contains actual code:

| Offset | Source / Type                        |                      |
|--------|--------------------------------------|----------------------|
| 0x000  | Sync -- from Current EL with SP_EL0  |                      |
| 0x080  | IRQ -- from Current EL with SP_EL0   |                      |
| 0x100  | FIQ -- from Current EL with SP_EL0   |                      |
| 0x180  | SError-- from Current EL with SP_EL0 |                      |
| 0x200  | Sync -- from Current EL with SP_ELx  |                      |
| 0x280  | IRQ -- from Current EL with SP_ELx   | <== KERNEL IRQ ENTRY |
| 0x300  | FIQ -- from Current EL with SP_ELx   |                      |
| 0x380  | SError                               |                      |
| 0x400  | Sync -- from Lower EL (AArch64)      | <== user space sync  |
| 0x480  | IRQ -- from Lower EL (AArch64)       | <== user space IRQ   |
| ...    |                                      |                      |
| 0x600  | Sync -- from Lower EL (AArch32)      |                      |
| ...    |                                      |                      |

VBAR\_EL1 is set during kernel boot (`__primary_switched` in `arch/arm64/kernel/head.S`). The IRQ entry (`el1_irq / el0_irq`) saves all registers to the kernel stack as a `pt_regs` structure, then calls `handle_arch_irq` — a function pointer set by the GIC driver.

## 1.4 GIC — Generic Interrupt Controller

| Version | Usage              | Notes                                    |
|---------|--------------------|------------------------------------------|
| GICv1   | Legacy, ARMv7      | Rare in modern kernels                   |
| GICv2   | ARMv7/early ARMv8  | Most Cortex-A53/A57 era                  |
| GICv3   | Modern ARMv8/ARMv9 | Cortex-A55/A75+, supports >8 CPUs        |
| GICv4   | GICv3 + virt       | Direct virtual interrupt injection (KVM) |

### GIC Architecture (GICv2)

Distributor (GICD) — global, one per system: manages all SPIs (Shared Peripheral Interrupts); controls enable/disable, priority, target CPU, trigger type. MMIO registers: `GICD_CTLR`, `GICD_ISENBLER`, `GICD_IPRIORITYR`, `GICD_ITARGETSR`, `GICD_ICFGR`.

CPU Interface (GICC) — per-CPU: each CPU core has its own GICC; handles interrupt acknowledgment (`GICC_IAR`), EOI (`GICC_EOIR`), priority masking (`GICC_PMR`). In GICv3, GICC is replaced by system registers (`ICC_IAR1_EL1`, `ICC_EOIR1_EL1`, etc.).

GICv3 adds: Redistributor (GICR) — per-CPU, handles SGIs/PPIs locally; ITS (Interrupt Translation Service) — for MSI/MSI-X (PCIe); LPI (Locality-specific Peripheral Interrupts) — message-based, huge interrupt ID space.

### Interrupt Types

| Type                     | ID Range | Description                                   |
|--------------------------|----------|-----------------------------------------------|
| SGI (Software Generated) | 0–15     | IPI — inter-processor interrupts              |
| PPI (Private Peripheral) | 16–31    | Per-CPU: arch timer (PPI 30), PMU             |
| SPI (Shared Peripheral)  | 32–1019  | All external peripherals (UART, GPIO, DMA...) |
| LPI                      | >=8192   | MSI-based (GICv3+, PCIe)                      |

### Interrupt Flow Through GIC

```

Peripheral asserts interrupt line
|
GICD receives it, checks: enabled? priority? target CPU?
|
GICD forwards to target CPU's GICC/GICR
|
GICC raises nIRQ signal to CPU core
|
CPU core takes IRQ exception => jumps to vector table
|
Kernel IRQ handler reads GICC_IAR (ACK) => gets hwirq number
|
Kernel dispatches to device driver ISR
|
Kernel writes GICC_EOIR (End Of Interrupt)

```

The ACK (IAR read) is critical — it deactivates the interrupt in the GIC and returns the interrupt ID. Without EOI, the GIC will not forward that interrupt again.

## 1.5 Linux IRQ Domains

### The Problem IRQ Domains Solve

A modern SoC has multiple interrupt controllers in a hierarchy: GIC (root) → GPIO controller → individual GPIO pins; GIC (root) → PCIe controller → MSI interrupts; GIC (root) → secondary interrupt controller (e.g., PMIC).

Each controller has its own hardware interrupt numbering (hwirq). The kernel needs a way to map hwirq → Linux virq (virtual IRQ number, what drivers use). IRQ domains provide this mapping per interrupt controller.

### IRQ Domain Structure

```
struct irq_domain {
    struct list_head link;
    const char *name;
    const struct irq_domain_ops *ops;
    void *host_data;           // pointer to controller-specific data
    unsigned int flags;
    struct fwnode_handle *fwnode; // DT node or ACPI handle
    irq_hw_number_t hwirq_max;
    unsigned int revmap_size;
    struct radix_tree_root revmap_tree; // hwirq -> virq mapping
    unsigned int linear_revmap[]; // for small, dense hwirq spaces
};
```

### IRQ Domain Ops

```
struct irq_domain_ops {
    int (*match)(struct irq_domain *d, struct device_node *node,
                 enum irq_domain_bus_token bus_token);
    int (*map)(struct irq_domain *d, unsigned int virq,
               irq_hw_number_t hw);
    void (*unmap)(struct irq_domain *d, unsigned int virq);
    int (*xlate)(struct irq_domain *d, struct device_node *node,
                  const u32 *intspec, unsigned int intsize,
                  unsigned long *out_hwirq, unsigned int *out_type);
    int (*alloc)(struct irq_domain *d, unsigned int virq,
                  unsigned int nr_irqs, void *arg);
    void (*free)(struct irq_domain *d, unsigned int virq,
                  unsigned int nr_irqs);
};
```

xlate — parses DT interrupts property into hwirq + trigger type. map — called when a virq is allocated; sets up irq\_chip and irq\_data. alloc/free — used in hierarchical domains (GICv3 + ITS).

### IRQ Domain Types

| Type                  | Use Case                                      |
|-----------------------|-----------------------------------------------|
| IRQ_DOMAIN_MAP_LINEAR | Dense hwirq space (GPIO banks, GIC SPIs)      |
| IRQ_DOMAIN_MAP_TREE   | Sparse hwirq space (LPis, MSIs)               |
| IRQ_DOMAIN_MAP_NOMAP  | virq == hwirq (legacy x86-style, rare on ARM) |
| Hierarchical          | GICv3 + ITS, PCIe MSI chains                  |

### Hierarchical IRQ Domains (GICv3 + ITS)

For PCIe MSI on GICv3: PCIe device → ITS domain → GIC domain. Each level has its own `irq_domain`. The `irq_data` for a `virq` forms a chain:

```
struct irq_data {
    u32 mask;
    unsigned int irq;          // virq
    irq_hw_number_t hwirq;     // hwirq in this domain
    struct irq_data *parent_data; // parent domain's irq_data
    struct irq_domain *domain;
    struct irq_chip *chip;
    ...
};
```

## 1.6 irq\_chip — The Hardware Abstraction

```
struct irq_chip {
    const char *name;
    void (*irq_mask)(struct irq_data *data);
    void (*irq_unmask)(struct irq_data *data);
    void (*irq_ack)(struct irq_data *data);
    void (*irq_eoi)(struct irq_data *data);
    int (*irq_set_type)(struct irq_data *data, unsigned int flow_type);
    int (*irq_set_affinity)(struct irq_data *data,
                           const struct cpumask *dest, bool force);
    void (*irq_enable)(struct irq_data *data);
    void (*irq_disable)(struct irq_data *data);
    ...
};
```

The GIC driver (`drivers/irqchip/irq-gic.c` for GICv2, `irq-gic-v3.c` for GICv3) implements this for GIC registers. A GPIO expander driver implements it for its own registers.

## 1.7 Complete Interrupt Flow in Linux/ARM64

Here is the full path from hardware to driver:

1. Peripheral (e.g., UART) asserts SPI line to GIC
2. GIC Distributor => CPU Interface => nIRQ to CPU core
3. CPU takes IRQ exception:
  - Hardware saves PC => ELR\_EL1, PSTATE => SPSR\_EL1
  - Jumps to VBAR\_EL1 + 0x280 (EL1h IRQ vector)
4. `arch/arm64/kernel/entry.S: ell_irq`
  - `kernel_entry 1` // save all regs to pt\_regs on stack
  - `bl handle_arch_irq` // call GIC's handler
5. GIC handler (`gic_handle_irq`):
  - Read ICC\_IAR1\_EL1 // ACK: get hwirq number
  - Call `handle_domain_irq(gic_domain, hwirq, regs)`
6. `handle_domain_irq`:
  - `irq_find_mapping(domain, hwirq) => virq`
  - `generic_handle_irq(virq)`
7. `generic_handle_irq => irq_desc[virq].handle_irq`
  - For edge: `handle_edge_irq`
  - For level: `handle_level_irq`
  - These call `action->handler` (the driver's ISR)
8. Driver ISR runs (e.g., `uart_interrupt()`)
  - Reads/clears hardware status
  - Wakes up waiting threads / queues work
9. Return from ISR => `irq_chip.irq_eoi()` => writes `ICC_EOIR1_EL1`

```

10. el1_irq epilogue:
    - kernel_exit 1          // restore pt_regs
    - eret                  // restore ELR_EL1 => PC, SPSR_EL1 => PSTATE

```

## 1.8 Device Tree Integration

```

// GIC definition (root interrupt controller)
gic: interrupt-controller@fee00000 {
    compatible = "arm,gic-v3";
    #interrupt-cells = <3>;
    interrupt-controller;
    reg = <0x0 0xfe00000 0x0 0x10000>, // GICD
        <0x0 0xfef00000 0x0 0xc0000>; // GICR
};

// UART using GIC SPI
uart0: serial@ff180000 {
    compatible = "snps,dw-apb-uart";
    reg = <0x0 0xff180000 0x0 0x100>;
    interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
    //           ^type ^hwirq ^trigger
    interrupt-parent = <&gic>;
};

// GPIO controller as secondary interrupt controller
gpio0: gpio@ff720000 {
    compatible = "rockchip,gpio-bank";
    interrupt-controller;
    #interrupt-cells = <2>;
    interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-parent = <&gic>;
};

// Device using GPIO interrupt
button {
    interrupt-parent = <&gpio0>;
    interrupts = <5 IRQ_TYPE_EDGE_FALLING>; // GPIO pin 5
};

```

## 1.9 Key Kernel Files (ARM64)

| File                         | Purpose                                                       |
|------------------------------|---------------------------------------------------------------|
| arch/arm64/kernel/entry.S    | Exception vectors, el1_irq, el0_irq, kernel_entry/exit macros |
| arch/arm64/kernel/irq.c      | handle_arch_irq, set_handle_irq                               |
| drivers/irqchip/irq-gic-v3.c | GICv3 driver, gic_handle_irq, domain ops                      |
| kernel/irq/irqdomain.c       | IRQ domain core: irq_domain_add_*, irq_create_mapping         |
| kernel/irq/chip.c            | irq_chip helpers, handle_edge_irq, handle_level_irq           |
| kernel/irq/manage.c          | request_irq, free_irq, threaded IRQ support                   |

## 1.10 SMP Considerations

- ❑ SGIs (IDs 0–15) are used for IPIs: IPI\_RESCHEDULE, IPI\_CALL\_FUNC, IPI\_CPU\_STOP, etc.
- ❑ GICv3 uses ICC\_SGI1R\_EL1 system register to send SGIs (no MMIO needed)

- `irq_set_affinity()` routes SPIs to specific CPUs via `GICD_IROUTER` (GICv3) or `GICD_ITARGETSR` (GICv2)
- `/proc/irq/<N>/smp_affinity` exposes this to userspace
- IRQ balancing: `irqbalance` daemon or manual affinity for performance-critical drivers (NIC, storage)

## 1.11 FIQ on ARM

FIQ (Fast Interrupt) has lower latency because: dedicated banked registers `r8–r14` in ARM32 (no save/restore needed); separate vector entry. In Linux, FIQ is used for secure monitor calls (TrustZone), watchdog NMI, or hardware performance counters — not general device interrupts. GICv3 can route interrupts as Group 0 (FIQ at EL1) or Group 1 (IRQ at EL1).

# 2. Complete SPI Interrupt Flow — Shared Interrupt Routing

## 2.1 What is an SPI in GIC Context?

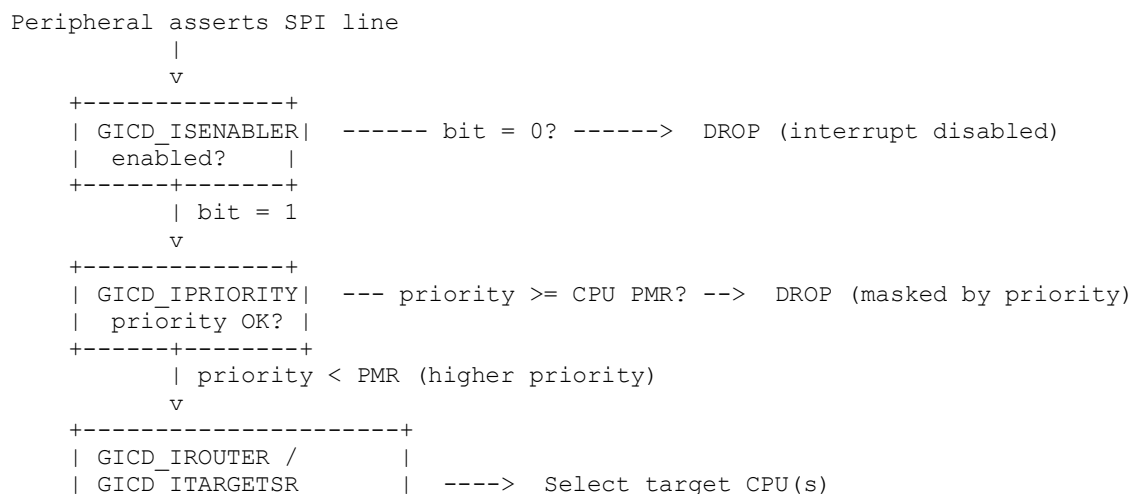
SPI = Shared Peripheral Interrupt (GIC terminology, NOT the SPI bus protocol). `hwirq` IDs: 32–1019 in GICv2/v3. "Shared" means the interrupt line is routed through the GIC Distributor and can target any CPU. One GIC SPI line = one peripheral's interrupt signal.

## 2.2 Hardware Path — Peripheral to GIC to CPU

### GIC Distributor Registers

| Register                        | Purpose                               |
|---------------------------------|---------------------------------------|
| <code>GICD_ISENABLER[n]</code>  | Enable bit for each SPI               |
| <code>GICD_IPRIORITYR[n]</code> | Priority (0=highest, 255=lowest)      |
| <code>GICD_ITARGETSR[n]</code>  | Target CPU mask (GICv2)               |
| <code>GICD_IROUTER[n]</code>    | Target CPU affinity (GICv3)           |
| <code>GICD_ICFGR[n]</code>      | Edge (0b10) or Level (0b00) triggered |
| <code>GICD_ISPENDR[n]</code>    | Set pending state                     |

### GIC Decision Flow Diagram



```

| which CPU? |
+-----+-----+
|
| v
+-----+
| GICC/GICR | ----> Assert nIRQ on target CPU core
+-----+

```

## 2.3 CPU Exception Entry (ARM64)

### ARM64 Exception Vector Table Layout

```

VBAR_El1 base address (512-byte aligned)
|
+-- +0x000 [Sync | Current EL | SP_El0]
+-- +0x080 [IRQ | Current EL | SP_El0]
+-- +0x100 [FIQ | Current EL | SP_El0]
+-- +0x180 [SError| Current EL | SP_El0]
|
+-- +0x200 [Sync | Current EL | SP_ElX] <== kernel sync fault
+-- +0x280 [IRQ | Current EL | SP_ElX] <== KERNEL IRQ ENTRY (el1_irq)
+-- +0x300 [FIQ | Current EL | SP_ElX]
+-- +0x380 [SError| Current EL | SP_ElX]
|
+-- +0x400 [Sync | Lower EL | AArch64] <== user syscall/fault
+-- +0x480 [IRQ | Lower EL | AArch64] <== USER IRQ ENTRY (el0_irq)
+-- +0x500 [FIQ | Lower EL | AArch64]
+-- +0x580 [SError| Lower EL | AArch64]
|
+-- +0x600 [Sync | Lower EL | AArch32]
+-- +0x680 [IRQ | Lower EL | AArch32]
+-- +0x700 [FIQ | Lower EL | AArch32]
+-- +0x780 [SError| Lower EL | AArch32]

```

Each slot = 128 bytes of actual code (not just a branch instruction)

### CPU Exception Entry — pt\_regs on Stack

Before IRQ:

After kernel\_entry (pt\_regs on stack):

```

SP_El1 ----> +-----+
| kernel stack |
| (running    |
| code)       |
+-----+

ELR_El1 = interrupted PC
SPSR_El1 = interrupted PSTATE

SP_El1 ----> +-----+ <-- sp
| x0          |
| x1 .. x29   |
| x30 (lr)    |
| sp_el0      |
| ELR_El1     | <-- saved PC
| SPSR_El1    | <-- saved PSTATE
+-----+
struct pt_regs

```

### el1\_irq Handler in entry.S

```

SYM_CODE_START_LOCAL(el1_irq)
    kernel_entry 1          // Save ALL registers to pt_regs on stack
    enable_da_f            // Re-enable Debug/SError, keep IRQ masked

    mov x0, sp              // x0 = pointer to pt_regs
    bl handle_arch_irq      // Call C function (GIC handler)

    kernel_exit 1           // Restore all registers from pt_regs
    eret                   // ELR_El1 -> PC, SPSR_El1 -> PSTATE
SYM_CODE_END(el1_irq)

```

## 2.4 GIC Driver Handler

### gic\_handle\_irq — ACK and Dispatch

```
static void __exception_irq_entry gic_handle_irq(struct pt_regs *regs)
{
    u32 irqnr;

    irqnr = gic_read_iar();    // Read ICC_IAR1_EL1 => ACK interrupt
                              // Returns hwirq number (e.g., 64 for SPI 32)
                              // Also DEACTIVATES the interrupt in GIC

    if (likely(irqnr > 15 && irqnr < 1020)) {
        // Regular SPI or PPI (not SGI)
        handle_domain_irq(gic_data.domain, irqnr, regs);
        return;
    }
    if (irqnr < 16) {
        // SGI -- inter-processor interrupt
        __handle_domain_irq(gic_data.domain, irqnr, false, regs);
        gic_write_eoir(irqnr);
        return;
    }
    // irqnr == 1023: spurious interrupt, ignore
}
```

Key point: Reading ICC\_IAR1\_EL1 (GICv3) simultaneously: (1) Acknowledges the interrupt, (2) Returns the hwirq number, (3) Deactivates the interrupt in GIC.

## 2.5 IRQ Domain Lookup — hwirq to virq

```
int handle_domain_irq(struct irq_domain *domain,
                     unsigned int hwirq, struct pt_regs *regs)
{
    unsigned int irq;

    irq_enter();                // Enter interrupt context

    irq = irq_find_mapping(domain, hwirq); // hwirq -> virq lookup
    if (likely(irq))
        generic_handle_irq(irq);        // Dispatch to virq handler

    irq_exit();                 // Exit, run softirqs if pending
    return ret;
}

// irq_find_mapping: O(1) for linear map, tree for LPIs
unsigned int irq_find_mapping(struct irq_domain *domain,
                             irq_hw_number_t hwirq)
{
    if (hwirq < domain->revmap_size)
        return domain->linear_revmap[hwirq]; // O(1) direct lookup

    data = radix_tree_lookup(&domain->revmap_tree, hwirq);
    return data ? data->irq : 0;
}
```

## 2.6 irq\_desc and Flow Handler

struct irq\_desc is the central per-interrupt descriptor:

```
struct irq_desc {
    struct irq_data irq_data;        // hwirq, domain, chip, affinity
    irq_flow_handler_t handle_irq;   // flow handler (edge/level/etc.)
    struct irqaction *action;        // linked list of ISR handlers
    unsigned int depth;              // disable depth (0 = enabled)
    unsigned int irq_count;          // stats
    spinlock_t lock;
    const char *name;
    ...
};
```

## Flow Handler: Level vs Edge

```

LEVEL-TRIGGERED (handle_level_irq):
-----
  IRQ fires
  |
  v
  mask_ack_irq()          <- MASK at GIC (GICD_ICENABLER bit)
                           Prevents re-entry while ISR runs
  |
  v
  handle_irq_event()      <- Call all irqaction handlers
  |
  v
  cond_unmask_irq()       <- UNMASK at GIC (GICD_ISENABLER bit)
  |
  v
  If peripheral still asserts line -> GIC re-triggers immediately

EDGE-TRIGGERED (handle_edge_irq):
-----
  Edge detected -> No masking needed (edge is self-clearing in GIC)
  |
  v
  handle_irq_event()      <- Call all irqaction handlers
  |
  +-- If another edge arrives DURING handling:
  |     mark IRQS_PENDING, mask temporarily
  |
  v
  After ISR: check IRQS_PENDING -> resend if needed

```

## 2.7 How Shared Interrupts Work

In Linux, multiple drivers can share the same virq by passing `IRQF_SHARED` to `request_irq()`. This is software-level sharing where multiple irqaction entries are chained on the same `irq_desc`.

```

irq_desc[virq]
+-- action -> irqaction (driver A, handler_A, IRQF_SHARED)
                    +-- next -> irqaction (driver B, handler_B, IRQF_SHARED)
                        +-- next -> NULL

```

### handle\_irq\_event\_percpu — Shared IRQ Demultiplexer

```

irqreturn_t handle_irq_event_percpu(struct irq_desc *desc)
{
    irqreturn_t retval = IRQ_NONE;
    struct irqaction *action;

    // Walk the linked list of ALL registered handlers for this virq
    for_each_action_of_desc(desc, action) {
        irqreturn_t res;
        res = action->handler(desc->irq_data.irq, action->dev_id);
        //    ^ Call each driver's ISR with its own dev_id

        switch (res) {
            case IRQ_WAKE_THREAD:
                __irq_wake_thread(desc, action); // Wake threaded handler
            case IRQ_HANDLED:
                retval = IRQ_HANDLED;
                break;
            case IRQ_NONE:
                break; // This driver didn't cause the interrupt
        }
    }

    if (retval == IRQ_NONE)
        note_interrupt(desc, IRQ_NONE); // spurious interrupt warning
}

```

```

    return retval;
}

```

## IRQ Return Values

| Return Value    | Meaning                                                                   |
|-----------------|---------------------------------------------------------------------------|
| IRQ_NONE        | This driver did NOT cause the interrupt; kernel continues to next handler |
| IRQ_HANDLED     | This driver handled it; but all handlers are still called for IRQF_SHARED |
| IRQ_WAKE_THREAD | Handled minimally in hardirq context; wake the threaded handler           |

Important: For IRQF\_SHARED, the kernel calls ALL handlers in the chain regardless of IRQ\_HANDLED. It only uses the return value for spurious interrupt detection.

## 2.8 Shared Interrupt Routing — Detailed Example

Suppose two drivers share virq 45 (both called request\_irq(45, handler, IRQF\_SHARED, ...)):

```

irq_desc
+-- irq_data.hwirq = 77  (GIC SPI 45 = hwirq 77)
+-- handle_irq = handle_level_irq
+-- action:
    irqaction: handler=spi_bus_irq,  dev_id=&spi_master,  IRQF_SHARED
        +-- next:
    irqaction: handler=uart_irq,      dev_id=&uart_port,   IRQF_SHARED
        +-- next: NULL

```

When the interrupt fires:

1. GIC delivers hwirq=77 -> irq\_find\_mapping -> virq=45
2. handle\_level\_irq called
3. handle\_irq\_event\_percpu iterates:

```

Call spi_bus_irq(45, &spi_master):
  reads SPI_STATUS register
  SPI_INT_PENDING bit = 0
  return IRQ_NONE  <- "not me"

```

```

Call uart_irq(45, &uart_port):
  reads UART_STATUS register
  UART_RX_READY bit = 1
  processes RX data
  clears UART_RX_READY bit
  return IRQ_HANDLED  <- "I handled it"

```

4. retval = IRQ\_HANDLED -> no spurious warning
5. EOI sent to GIC

## 2.9 EOI Flow and GICv3 Priority Drop

After all ISR handlers complete:

```

|
v
irq_chip->irq_eoi()  [GICv3: gic_eoi_irq()]
gic_write_eoir(hwirq)
|
Write ICC_EOIR1_EL1

```

Non-split mode (default):

```

  ICC_EOIR1_EL1 write
  +-- Priority DROP  (CPU can now take lower-priority interrupts)

```

```

+-- DEACTIVATE      (GIC can forward this SPI again if still asserted)

Split mode (virtualization/KVM):
  ICC_EOIR1_EL1 -> Priority DROP only
  ICC_DIR_EL1   -> DEACTIVATE separately

```

## 2.10 irq\_exit and Softirq Processing

```

void irq_exit(void)
{
    preempt_count_sub(HARDIRQ_OFFSET); // Decrement hardirq count

    if (!in_interrupt() && local_softirq_pending())
        invoke_softirq(); // Run pending softirqs:
                          //   HI_SOFTIRQ, TIMER_SOFTIRQ,
                          //   NET_TX/RX_SOFTIRQ, BLOCK_SOFTIRQ,
                          //   TASKLET_SOFTIRQ, SCHED_SOFTIRQ,
                          //   RCU_SOFTIRQ
}

```

## 2.11 Registering a Shared IRQ — Driver Side

```

// In driver probe:
ret = request_irq(irq,                // virq from platform_get_irq()
                 my_irq_handler,      // ISR function
                 IRQF_SHARED,         // Allow sharing
                 "my-device",         // Name in /proc/interrupts
                 dev);                // dev_id MUST be unique per driver
                                     // Used to identify handler on free_irq()

// ISR must check own hardware:
static irqreturn_t my_irq_handler(int irq, void *dev_id)
{
    struct my_device *dev = dev_id;
    u32 status = readl(dev->base + STATUS_REG);

    if (!(status & MY_INT_BIT))
        return IRQ_NONE; // Not my interrupt

    writel(status, dev->base + STATUS_REG); // Clear interrupt
    // ... handle event ...
    return IRQ_HANDLED;
}

// In driver remove:
free_irq(irq, dev); // dev_id used to find and remove the correct irqaction

```

## 2.12 /proc/interrupts — What You See

|       | CPU0    | CPU1 | CPU2 | CPU3 |       |       |         |                |
|-------|---------|------|------|------|-------|-------|---------|----------------|
| 45:   | 1234    | 567  | 890  | 123  | GICv3 | 33    | Level   | uart-pl011     |
| 46:   | 0       | 0    | 0    | 0    | GICv3 | 34    | Edge    | dma-controller |
| 47:   | 5678    | 0    | 0    | 0    | GICv3 | 35    | Level   | spi-bus, uart- |
| pl011 |         |      |      |      |       |       |         |                |
| ^     | ^       |      |      |      | ^     | ^     | ^       | ^              |
| virq  | per-CPU |      |      |      | chip  | hwirq | trigger | driver name(s) |
|       | counts  |      |      |      |       |       |         |                |

Note: virq 47 shows TWO driver names => IRQF\_SHARED, two handlers registered

## 3. Threaded IRQs

### 3.1 Why Threaded IRQs Exist

Traditional hardirq handlers run with interrupts disabled. This means: you cannot sleep, block, or acquire a mutex; you cannot call any function that might schedule; you must be extremely fast (microseconds).

But many drivers need to: acquire a mutex (I2C/SPI bus lock); sleep waiting for hardware; do significant data processing. Threaded IRQs solve this by splitting the handler into two parts:

- Hardirq (top half): runs in interrupt context, does minimal work, wakes a thread
- Thread (bottom half): runs in a kernel thread (process context), can sleep/block

### 3.2 Two Ways to Use Threaded IRQs

#### Method 1: request\_threaded\_irq()

```
int request_threaded_irq(unsigned int irq,
                        irq_handler_t handler,          // hardirq (top half)
                        irq_handler_t thread_fn,        // threaded (bottom half)
                        unsigned long flags,
                        const char *name,
                        void *dev_id);
```

#### Method 2: request\_irq() with IRQF\_ONESHOT

```
// If handler=NULL, kernel provides a default that just wakes the thread
request_irq(irq, NULL, IRQF_ONESHOT, "my-device", dev);
// OR
request_threaded_irq(irq, NULL, my_thread_fn, IRQF_ONESHOT, "my-device", dev);
```

### 3.3 IRQF\_ONESHOT — The Critical Flag

IRQF\_ONESHOT means: Keep the interrupt line MASKED until the threaded handler completes.

Without it, for level-triggered interrupts:

```
hardirq runs -> returns IRQ_WAKE_THREAD -> unmask IRQ
-> peripheral still asserts line -> IRQ fires AGAIN immediately
-> infinite interrupt storm before thread even runs
```

With IRQF\_ONESHOT:

```
hardirq runs -> returns IRQ_WAKE_THREAD -> IRQ stays MASKED
-> thread runs -> clears hardware -> thread exits
-> kernel unmask IRQ -> safe to re-trigger
```

**RULE:** Always use IRQF\_ONESHOT with threaded IRQs for level-triggered interrupts.

### 3.4 Threaded IRQ Complete Flow Diagram

```
[HARDWARE]
Peripheral asserts SPI line -> GIC -> CPU exception

[HARDIRQ CONTEXT - interrupts disabled, cannot sleep]
+-----+
| ell_irq -> gic_handle_irq -> handle_domain_irq |
| -> handle_level_irq -> handle_irq_event_percpu |
| |
| Call hardirq handler (top half): |
|   irqreturn_t my_hardirq(int irq, void *dev_id) |
+-----+
```

```

|      {
|          // Minimal work only:
|          // - Read/clear interrupt status register
|          // - Disable further interrupts from device
|          return IRQ_WAKE_THREAD;  <- wake the thread
|      }
|-----+
|          | IRQ_WAKE_THREAD
|          v
|-----+
|  __irq_wake_thread(desc, action)
|  wake_up_process(action->thread)  <- wake irq/N thread
|
|  IRQF_ONESHOT: IRQ line stays MASKED
|-----+
|
|          v
| [PROCESS CONTEXT - kernel thread, CAN sleep/block]
|-----+
|  Kernel thread: "irq/45-my-device"  (kthread)
|
|  irq_thread() [kernel/irq/manage.c]:
|  while (!kthread_should_stop()) {
|      if (!irq_wait_for_interrupt(action)) break;
|
|      // Run the threaded handler:
|      action->thread_fn(irq, action->dev_id)
|      |
|      my_thread_fn():
|          mutex_lock(&bus->lock);    <- CAN sleep here!
|          i2c_transfer(...);        <- CAN block here!
|          process_data();
|          mutex_unlock(&bus->lock);
|          return IRQ_HANDLED;
|
|      irq_finalize_oneshot(desc, action)
|          -> unmask IRQ  <- NOW safe to re-trigger
|  }
|-----+

```

### 3.5 Hardirq vs Thread Context — What You Can/Cannot Do

| Operation               | Hardirq Context | Thread Context |
|-------------------------|-----------------|----------------|
| in_interrupt()          | true            | false          |
| IRQs state              | disabled/masked | enabled        |
| readl/writel            | [YES]           | [YES]          |
| spin_lock_irqsave       | [YES]           | [YES]          |
| atomic operations       | [YES]           | [YES]          |
| wake_up()               | [YES]           | [YES]          |
| mutex_lock              | [NO]            | [YES]          |
| sleep / msleep          | [NO]            | [YES]          |
| kmalloc(GFP_KERNEL)     | [NO]            | [YES]          |
| i2c_transfer / spi_sync | [NO]            | [YES]          |
| copy_to_user            | [NO]            | [YES]          |
| schedule()              | [NO]            | [YES]          |

### 3.6 irq\_thread Kernel Thread Lifecycle

```
kthread_create(irq_thread, ...)
```

```

|
v
+-----+
| irq_thread_check_affinity() <- set CPU affinity |
| while (!kthread_should_stop()): |
| +-----+ |
| | irq_wait_for_interrupt(action) |
| | set_current_state(TASK_INTERRUPTIBLE) |
| | if (test_and_clear_bit(IRQTF_RUNTHREAD, |
| | &action->thread_flags |
| | break; <- woken by hardirq |
| | schedule(); <- sleep until woken |
| +-----+ |
| set_current_state(TASK_RUNNING) |
| action->thread_fn(irq, dev_id) <- run handler |
| irq_finalize_oneshot() <- unmask if IRQF_ONESHOT |
+-----+

```

Thread name: "irq/<virq>-<name>"  
Scheduling: SCHED\_FIFO priority 50 (real-time)

View with: ps aux | grep "irq/"

### 3.7 IRQF\_ONESHOT Masking Sequence (Level-Triggered)

```

Time ----->

Peripheral: =====
            (asserts level HIGH throughout)

IRQ masked: ..... (masked) ..... (unmasked)

Hardirq:    +-----+                               +-----+
            | top half |                               | top half |
            | runs      |                               | runs      |
            +-----+                               +-----+
            | IRQ_WAKE_THREAD |                               |
            | MASK IRQ (IRQF_ONESHOT) |                               |
            v                                         |
Thread:     +-----+                               |
            | thread_fn runs |                               |
            | mutex_lock    |                               |
            | i2c_transfer (blocks) |                               |
            | clears hardware |                               |
            | mutex_unlock   |                               |
            | return IRQ_HANDLED |                               |
            +-----+                               |
            | irq_finalize_oneshot |                               |
            | UNMASK IRQ -----> |                               |
            |                               |                               |
            |                               |                               |
                                   IRQ fires again

```

### 3.8 Without IRQF\_ONESHOT — The Interrupt Storm

```

Peripheral: =====
            (level HIGH, hardware not yet cleared)

Hardirq:    +---+---+---+---+---+---+---+---+
            | || || || || || || || | <- fires continuously!
            +---+---+---+---+---+---+---+---+
                                   Thread never gets CPU!
            (IRQ unmasked immediately after hardirq -> re-fires)
            <- INTERRUPT STORM -> system hangs

```

### 3.9 Practical Driver Example: I2C Touchscreen

A touchscreen connected via I2C is a perfect use case — the interrupt fires when touch data is ready, but reading the data requires I2C transfers (which can sleep):

```
static irqreturn_t ts_hardirq(int irq, void *dev_id)
{
    struct ts_data *ts = dev_id;
    disable_irq_nosync(irq);    // optional: prevent re-entry
    return IRQ_WAKE_THREAD;    // wake the thread
}

static irqreturn_t ts_thread_fn(int irq, void *dev_id)
{
    struct ts_data *ts = dev_id;
    u8 buf[6];

    // CAN sleep here -- we're in process context
    i2c_master_recv(ts->client, buf, sizeof(buf)); // blocks until I2C done

    // Process touch data
    input_report_abs(ts->input, ABS_X, (buf[0] << 8) | buf[1]);
    input_report_abs(ts->input, ABS_Y, (buf[2] << 8) | buf[3]);
    input_report_key(ts->input, BTN_TOUCH, buf[4] & 0x01);
    input_sync(ts->input);

    return IRQ_HANDLED;
    // After return: kernel unmask IRQ (IRQF_ONESHOT)
}

static int ts_probe(struct i2c_client *client)
{
    ret = request_threaded_irq(
        client->irq,
        ts_hardirq,          // top half (can be NULL)
        ts_thread_fn,        // bottom half
        IRQF_TRIGGER_FALLING | IRQF_ONESHOT,
        "my-touchscreen",
        ts
    );
}
```

### 3.10 NULL Top Half — Kernel Default Handler

```
// If you pass NULL as the hardirq handler, kernel uses:
static irqreturn_t irq_default_primary_handler(int irq, void *dev_id)
{
    return IRQ_WAKE_THREAD;
}

// Simplest possible threaded IRQ:
request_threaded_irq(irq, NULL, my_thread_fn,
    IRQF_ONESHOT | IRQF_TRIGGER_FALLING,
    "my-device", dev);
```

### 3.11 Threaded IRQ vs Other Bottom Half Mechanisms

| Property        | Threaded IRQ      | Tasklet       | Workqueue         |
|-----------------|-------------------|---------------|-------------------|
| Context         | Process (kthread) | Softirq       | Process (kthread) |
| Can sleep?      | YES               | NO            | YES               |
| Can use mutex?  | YES               | NO            | YES               |
| Scheduling      | SCHED_FIFO 50     | After hardirq | Normal kthread    |
| Per-IRQ thread? | YES (dedicated)   | NO (shared)   | NO (pool)         |
| Latency         | Low (RT prio)     | Very low      | Higher            |

| Property     | Threaded IRQ     | Tasklet    | Workqueue     |
|--------------|------------------|------------|---------------|
| Use case     | I2C/SPI drivers  | Network RX | Deferred work |
| IRQF_ONESHOT | Required (level) | N/A        | N/A           |

### 3.12 Key Kernel Files

| File                      | Purpose                                                |
|---------------------------|--------------------------------------------------------|
| kernel/irq/manage.c       | request_threaded_irq, irq_thread, irq_finalize_oneshot |
| kernel/irq/handle.c       | handle_irq_event_percpu, __irq_wake_thread             |
| include/linux/interrupt.h | IRQF_* flags, request_threaded_irq prototype           |

## 4. I2C Driver Flow for STK3310

### 4.1 System Overview

```
+-----+
|          STK3310 I2C Driver Stack          |
|-----+-----+
| Userspace: /dev/iio:device0 or /sys/bus/iio/devices/... |
|               |                                     |
| IIO Subsystem: iio_device, iio_chan_spec, iio_info      |
|               |                                     |
| STK3310 Driver: stk3310_probe / stk3310_read_raw / stk3310_irq |
|               |                                     |
| I2C Core:      i2c_transfer / i2c_smbus_read_byte_data  |
|               |                                     |
| I2C Bus Driver: i2c_adapter (Qualcomm/DesignWare/etc.) |
|               |                                     |
| Hardware: I2C Controller MMIO <-> SDA/SCL lines <-> STK3310 |
+-----+
```

### 4.2 Hardware — STK3310 on I2C Bus

| Property      | Value                                                   |
|---------------|---------------------------------------------------------|
| I2C address   | 0x48 (ADDR pin low) or 0x49 (ADDR pin high)             |
| Interface     | I2C, up to 400 kHz (Fast Mode)                          |
| Interrupt     | Active-low, open-drain, connected to GPIO               |
| Registers     | 8-bit address, 8-bit data                               |
| Key registers | STATE(0x00), PSDATA_H(0x08), ALSDATA_H(0x0C), INT(0x25) |

### Device Tree Representation

```
// I2C controller node (e.g., Qualcomm QUP I2C)
i2c@78b6000 {
    compatible = "qcom,i2c-msm-v2";
    reg = <0x78b6000 0x600>;
```

```

interrupts = <GIC_SPI 96 IRQ_TYPE_LEVEL_HIGH>;
clocks = <&gcc GCC_BLSP1_QUP2_I2C_APPS_CLK>;
clock-frequency = <400000>; // 400 kHz Fast Mode
#address-cells = <1>;
#size-cells = <0>;

// STK3310 as child device
stk3310@48 {
    compatible = "sensortek,stk3310";
    reg = <0x48>; // I2C address
    interrupt-parent = <&tlmm>;
    interrupts = <43 IRQ_TYPE_EDGE_FALLING>;
    interrupt-names = "stk_irq";
    vdd-supply = <&pm8916_l17>;
};
};

```

## 4.3 I2C Controller (Adapter) — The Hardware Driver

### Key Structures

```

// The adapter represents ONE physical I2C bus
struct i2c_adapter {
    const struct i2c_algorithm *algo; // hardware transfer ops
    void *algo_data; // controller private data
    struct device dev;
    int nr; // bus number (/dev/i2c-N)
    struct mutex bus_lock; // serializes all transfers
    int timeout;
    int retries;
    ...
};

// The algorithm = hardware-specific transfer implementation
struct i2c_algorithm {
    int (*master_xfer)(struct i2c_adapter *adap,
                      struct i2c_msg *msgs, int num);
    int (*master_xfer_atomic)(struct i2c_adapter *adap,
                             struct i2c_msg *msgs, int num);
    u32 (*functionality)(struct i2c_adapter *adap);
};

```

### Controller Driver Probe Flow

```

platform_driver.probe (e.g., qcom_geni_i2c_probe)
|
+-- Get MMIO base: devm_ioremap_resource()
+-- Get IRQ: platform_get_irq()
+-- Get clocks: devm_clk_get()
+-- Enable clocks: clk_prepare_enable()
+-- Initialize hardware registers (baud rate, timing)
+-- request_irq() for controller interrupt
|
+-- Fill i2c_adapter:
|   adap->algo = &qcom_geni_i2c_algo
|   adap->dev.parent = &pdev->dev
|   adap->nr = bus_number
|
+-- i2c_add_numbered_adapter(&adap)
|
+-- Registers adapter with I2C core
    Scans DT children -> creates i2c_client for each

```

## 4.4 I2C Core — The Bus Abstraction Layer

### Core Data Structures

```
// Represents one device on the I2C bus
struct i2c_client {
    unsigned short addr;        // 7-bit I2C address (e.g., 0x48)
    char name[I2C_NAME_SIZE];   // "stk3310"
    struct i2c_adapter *adapter; // pointer to the bus controller
    struct device dev;          // embedded Linux device
    int irq;                    // virq from DT interrupt property
    ...
};

// i2c_msg -- The Transfer Unit
struct i2c_msg {
    __u16 addr;        // slave address (7-bit: 0x48 for STK3310)
    __u16 flags;        // I2C_M_RD for read, 0 for write
    __u16 len;          // number of bytes
    __u8 *buf;          // data buffer
};
```

### Core Transfer Path: i2c\_transfer()

```
int i2c_transfer(struct i2c_adapter *adap,
                 struct i2c_msg *msgs, int num)
{
    // 1. Check adapter supports master_xfer
    // 2. Acquire bus lock (serializes all transfers on this bus)
    i2c_lock_bus(adap, I2C_LOCK_SEGMENT);

    // 3. Retry loop (handles arbitration loss, NACK)
    ret = __i2c_transfer(adap, msgs, num);

    // 4. Release bus lock
    i2c_unlock_bus(adap, I2C_LOCK_SEGMENT);

    return ret;
}

int __i2c_transfer(struct i2c_adapter *adap,
                  struct i2c_msg *msgs, int num)
{
    for (tries = 0; tries < adap->retries; tries++) {
        ret = adap->algo->master_xfer(adap, msgs, num);
        if (ret != -EAGAIN)
            break; // -EAGAIN = arbitration lost, retry
    }
    return ret;
}
```

### How SMBus Read Translates to I2C Messages

```
i2c_smbus_read_byte_data(client, reg=0x08)
|
+-- builds two i2c_msg:
    msg[0]: addr=0x48, flags=WRITE, len=1, buf={0x08}  <- send register addr
    msg[1]: addr=0x48, flags=READ, len=1, buf={?}      <- read data byte
|
+-- i2c_transfer(adapter, msg, 2)
|
+-- adap->algo->master_xfer(adap, msg, 2)
|
+-- Hardware:
    START -> 0x90(W) -> ACK -> 0x08 -> ACK ->
    RESTART -> 0x91(R) -> ACK -> DATA -> NACK -> STOP
```

I2C Wire Protocol for register read:

|       |      |   |     |      |     |     |      |   |     |      |     |      |
|-------|------|---|-----|------|-----|-----|------|---|-----|------|-----|------|
| S     | 0x48 | W | A   | 0x08 | A   | Sr  | 0x48 | R | A   | DATA | N   | P    |
|       |      |   |     |      |     |     |      |   |     |      |     |      |
| Start | Addr | W | ACK | Reg  | ACK | Rst | Addr | R | ACK | Data | NAK | Stop |

## 4.5 I2C Client Registration Flow (Boot Time)

```

Kernel boot
|
v
i2c_add_numbered_adapter() [controller driver calls this]
|
+--- of_i2c_register_devices(adap)
    |
    +--- For each DT child node of I2C controller:
        of_i2c_register_device(adap, node)
        |
        +--- Read "reg" property -> addr = 0x48
        +--- Read "compatible" -> "sensortek,stk3310"
        +--- Read "interrupts" -> virq
        |
        +--- i2c_new_client_device(adap, &board_info)
            |
            +--- Allocate struct i2c_client
            +--- client->addr = 0x48
            +--- client->adapter = adap
            +--- client->irq = virq
            |
            +--- device_register(&client->dev)
                |
                +--- I2C bus match:
                    i2c_device_match()
                    -> of_driver_match_device()
                    -> "sensortek,stk3310" matches
                    -> i2c_device_probe()
                    -> stk3310_probe()

```

## 4.6 STK3310 Driver — Detailed Function Flow

### stk3310\_probe() Walkthrough

```

static int stk3310_probe(struct i2c_client *client,
                        const struct i2c_device_id *id)
{
    // Step 1: Allocate IIO device
    indio_dev = devm_iio_device_alloc(&client->dev, sizeof(*data));
    data = iio_priv(indio_dev);
    data->client = client;
    i2c_set_clientdata(client, indio_dev);

    // Step 2: Initialize regmap
    data->regmap = devm_regmap_init_i2c(client, &stk3310_regmap_config);

    // Step 3: Verify chip ID
    ret = stk3310_read_id(data); // reads STK3310_REG_ID (0x3E)
                                // expected: 0x13 (STK3310) or 0x1D (STK3311)

    // Step 4: Software reset + default config
    ret = stk3310_init(data);

    // Step 5: Setup IIO device
    mutex_init(&data->lock);
    indio_dev->info = &stk3310_info;
    indio_dev->name = STK3310_DRIVER_NAME;
    indio_dev->channels = stk3310_channels;
    indio_dev->modes = INDIO_DIRECT_MODE;

    // Step 6: Setup interrupt (if available)
    if (client->irq > 0) {
        ret = stk3310_setup_irq(indio_dev, client);
    }

    // Step 7: Register IIO device
    ret = devm_iio_device_register(&client->dev, indio_dev);
}

```

```

    return 0;
}

```

## stk3310\_init() — Hardware Initialization

```

static int stk3310_init(struct stk3310_data *data)
{
    // Software reset
    regmap_write(data->regmap, STK3310_REG_RESET, 0x01);
    msleep(10); // wait for reset

    // Set PS gain, integration time
    regmap_write(data->regmap, STK3310_REG_PSCTRL,
        STK3310_PS_GAIN_X1 | STK3310_PS_IT_0_391MS);

    // Set ALS gain, integration time
    regmap_write(data->regmap, STK3310_REG_ALSCTRL,
        STK3310_ALS_GAIN_X1 | STK3310_ALS_IT_50MS);

    // Set interrupt thresholds
    regmap_write(data->regmap, STK3310_REG_THDH2_PS, 0xA0); // high
    regmap_write(data->regmap, STK3310_REG_THDL2_PS, 0x20); // low

    // Enable PS + ALS
    data->state_reg = STK3310_STATE_EN_PS | STK3310_STATE_EN_ALS;
    return regmap_write(data->regmap, STK3310_REG_STATE, data->state_reg);
}

```

## stk3310\_read\_raw() — IIO Read Path

Called when userspace reads `/sys/bus/iio/devices/iio:device0/in_proximity_raw`:

```

static int stk3310_read_raw(struct iio_dev *indio_dev,
                           struct iio_chan_spec const *chan,
                           int *val, int *val2, long mask)
{
    struct stk3310_data *data = iio_priv(indio_dev);
    int ret;

    switch (mask) {
    case IIO_CHAN_INFO_RAW:
        mutex_lock(&data->lock);

        if (chan->type == IIO_PROXIMITY) {
            // Read 16-bit PS data: registers 0x08 (H) and 0x09 (L)
            ret = regmap_read(data->regmap,
                STK3310_REG_PS_DATA_H, val);
            // regmap_read internally calls:
            //   i2c_smbus_read_byte_data(client, reg)
            //   -> i2c_transfer(adapter, msgs, 2)
            //   -> master_xfer() -> hardware I2C transaction
        } else if (chan->type == IIO_LIGHT) {
            ret = regmap_read(data->regmap,
                STK3310_REG_ALS_DATA_H, val);
        }

        mutex_unlock(&data->lock);
        return IIO_VAL_INT;
    }
    return -EINVAL;
}

```

## regmap Call Chain

```

regmap_read(map, reg, val)
|
+-- Check cache: if cached and not volatile -> return cached value
|
+-- _regmap_raw_read(map, reg, val, 1)

```

```

|
+-- map->bus->read(map->bus_context, reg, val, 1)
|
+-- regmap_i2c_read()
|
+-- i2c_master_send() + i2c_master_recv()
|
+-- i2c_transfer()
|
+-- master_xfer() [hardware]

```

## 4.7 STK3310 Interrupt Handling

### IRQ Setup

```

static int stk3310_setup_irq(struct iio_dev *indio_dev,
                             struct i2c_client *client)
{
    // Register threaded IRQ
    ret = devm_request_threaded_irq(
        &client->dev,
        client->irq,
        stk3310_irq_handler,
        stk3310_irq_event_handler,
        IRQF_TRIGGER_FALLING | IRQF_ONESHOT,
        "stk3310_event",
        indio_dev
    );
    return ret;
}

```

### Hardirq Handler (Top Half)

```

static irqreturn_t stk3310_irq_handler(int irq, void *private)
{
    struct iio_dev *indio_dev = private;
    struct stk3310_data *data = iio_priv(indio_dev);

    // Minimal work: just record timestamp
    data->timestamp = iio_get_time_ns(indio_dev);

    return IRQ_WAKE_THREAD; // wake the threaded handler
}

```

### Threaded Handler (Bottom Half)

```

static irqreturn_t stk3310_irq_event_handler(int irq, void *private)
{
    struct iio_dev *indio_dev = private;
    struct stk3310_data *data = iio_priv(indio_dev);
    unsigned int flag_reg;

    mutex_lock(&data->lock); // CAN sleep here - process context

    // Read interrupt flag register (I2C read -- CAN sleep)
    regmap_read(data->regmap, STK3310_REG_FLAG, &flag_reg);

    if (flag_reg & STK3310_FLG_PS_INT_MASK) {
        // Proximity threshold crossed
        iio_push_event(indio_dev,
                       IIO_UNMOD_EVENT_CODE(IIO_PROXIMITY, 0,
                                             IIO_EV_TYPE_THRESH,
                                             IIO_EV_DIR_EITHER),
                       data->timestamp);
    }

    if (flag_reg & STK3310_FLG_ALS_INT_MASK) {
        // ALS threshold crossed -- push event to userspace
    }
}

```

```

        iio_push_event(indio_dev, ...);
    }

    // Clear interrupt flag
    regmap_write(data->regmap, STK3310_REG_FLAG,
                  flag_reg & ~(STK3310_FLG_PS_INT_MASK |
                               STK3310_FLG_ALS_INT_MASK));

    mutex_unlock(&data->lock);
    return IRQ_HANDLED;
    // After return: kernel unmask IRQ (IRQF_ONESHOT)
}

```

## 5. Tasklet and Workqueue Bottom Halves

### 5.1 Example 1: I2C Driver Using Tasklet (Softirq Bottom Half)

Use case: A simple I2C ADC (ADS1115 style) where the interrupt signals "conversion complete" and we need to read the result quickly. We use a tasklet — runs in softirq context, faster than a workqueue, but cannot sleep.

Rule: Tasklet runs after hardirq returns, in softirq context. It cannot sleep, cannot use mutex, but avoids the overhead of a full kernel thread.

#### Driver Structure

```

struct ads_data {
    struct i2c_client    *client;
    struct iio_dev       *indio_dev;
    struct tasklet_struct tasklet;    // bottom half
    spinlock_t          lock;        // spinlock (not mutex, tasklet can't sleep)
    u16                 raw_result;
    bool                conversion_done;
    wait_queue_head_t    wq;        // for blocking reads
};

```

#### Tasklet Registration in Probe

```

static int ads_probe(struct i2c_client *client, ...)
{
    indio_dev = devm_iio_device_alloc(&client->dev, sizeof(*data));
    data = iio_priv(indio_dev);
    data->client = client;

    // Initialize tasklet
    tasklet_init(&data->tasklet,
                ads_tasklet_func,    // bottom half function
                (unsigned long)data); // passed as arg

    spin_lock_init(&data->lock);
    init_waitqueue_head(&data->wq);

    // Register hardirq only (no thread_fn)
    devm_request_irq(&client->dev,
                    client->irq,
                    ads_hardirq,    // top half
                    IRQF_TRIGGER_FALLING,
                    "ads1115", data);

    devm_iio_device_register(&client->dev, indio_dev);
    return 0;
}

```

## Hardirq Handler — Schedules Tasklet

```
// TOP HALF (hardirq context) -- Runs with IRQs disabled, CANNOT sleep
static irqreturn_t ads_hardirq(int irq, void *dev_id)
{
    struct ads_data *data = dev_id;

    // Minimal work: schedule tasklet for actual I2C read
    tasklet_schedule(&data->tasklet);
    // ^ Sets TASKLET_STATE_SCHED bit
    // Adds tasklet to per-CPU tasklet_vec
    // Will run after this hardirq returns, in softirq context

    return IRQ_HANDLED;
}
```

## Tasklet Bottom Half

```
// BOTTOM HALF (softirq/tasklet context)
// Runs after hardirq, in softirq context
// CANNOT sleep, CANNOT use mutex, CAN use spinlock

static void ads_tasklet_func(unsigned long arg)
{
    struct ads_data *data = (struct ads_data *)arg;
    struct i2c_client *client = data->client;
    s32 result;
    unsigned long flags;

    // NOTE: For controllers with i2c_transfer_atomic() support:
    result = i2c_smbus_read_word_swapped(client, ADS_REG_CONVERSION);

    spin_lock_irqsave(&data->lock, flags);
    data->raw_result = (u16)result;
    data->conversion_done = true;
    spin_unlock_irqrestore(&data->lock, flags);

    // Wake up any blocking read
    wake_up_interruptible(&data->wq);
}
```

## Tasklet Execution Flow Diagram

[HARDWARE]  
ADS1115 ALERT/RDY pin -> GPIO -> GIC SPI -> CPU exception

[HARDIRQ CONTEXT - IRQs disabled]

```
+-----+
| ell_irq -> gic_handle_irq -> handle_edge_irq |
| -> handle_irq_event_percpu -> ads_hardirq() |
|     tasklet_schedule(&data->tasklet) |
|     +-- set TASKLET_STATE_SCHED bit |
|     +-- add to __tasklet_vec[cpu] (per-CPU list) |
|     return IRQ_HANDLED |
+-----+
```

| hardirq returns  
v

```
+-----+
| irq_exit() |
| preempt_count -= HARDIRQ_OFFSET |
| if local_softirq_pending(): |
|     invoke_softirq() |
|     +-- __do_softirq() |
|     +-- TASKLET_SOFTIRQ handler: |
|         tasklet_action() |
+-----+
```

|  
v

```
[SOFTIRQ CONTEXT - IRQs enabled, preemption disabled]
+-----+
| tasklet_action()                                |
|   for each tasklet in __tasklet_vec[cpu]:      |
|       if test_and_clear_bit(TASKLET_STATE_SCHED): |
|           set TASKLET_STATE_RUN                |
|           tasklet->func(tasklet->data)          |
|               +-- ads_tasklet_func(data)        |
|                   i2c_smbus_read_word_swapped() <- I2C read |
|                   spin_lock_irqsave()           |
|                   data->raw_result = result      |
|                   spin_unlock_irqrestore()       |
|                   wake_up_interruptible(&data->wq) |
|       clear TASKLET_STATE_RUN                  |
+-----+
|
v
[PROCESS CONTEXT - blocking read woken up]
ads_read_raw() was sleeping in wait_event
-> woken by wake_up_interruptible()
-> *val = data->raw_result
-> return IIO_VAL_INT
```

## Tasklet Timing on CPU Timeline

```
Normal code: #####
Hardirq:      ## (ads_hardirq: tasklet_schedule)
              |
irq_exit:     ## (invoke_softirq)
              |
Softirq:      #### (tasklet_action -> ads_tasklet_func)
              |
Normal code:  #####

Latency: hardirq -> tasklet = microseconds (same CPU, right after hardirq)
```

## WARNING: Tasklet + I2C

### IMPORTANT: Tasklet + I2C Warning

Tasklet context CANNOT sleep. Most I2C controllers use completion-based transfers: `i2c_transfer()` -> `wait_for_completion()` -> SLEEPS. Calling `i2c_transfer()` from tasklet context will trigger "BUG: scheduling while atomic" kernel warning or potentially deadlock.

**SAFE alternatives: (1) `i2c_transfer_atomic()` if controller supports it (uses polling), (2) Pre-read data in hardirq using MMIO, (3) Switch to workqueue (RECOMMENDED).**

## 5.2 Example 2: I2C Driver Using Workqueue (Process Context Bottom Half)

Use case: A real-world PMIC (Power Management IC) like MAX77686 connected via I2C. On interrupt, we need to: read multiple status registers (I2C, can sleep); acquire a mutex; potentially call regulator/clock framework APIs (can sleep). Workqueue is the right choice — it runs in process context, can sleep, can use mutex.

### Workqueue Styles

```
Style A: Shared system workqueue (schedule_work)
-----
Uses kernel's system_wq (shared among all drivers)
Simple, no allocation needed
Work may be delayed if system_wq is busy

Style B: Dedicated workqueue (alloc_workqueue + queue_work)
```

---

Driver creates its own workqueue thread  
 Guaranteed dedicated execution  
 Better for latency-sensitive or high-frequency IRQs  
 Used in: MMC, USB, network drivers

## Probe: Workqueue Setup

```
static int pmic_probe(struct i2c_client *client, ...)
{
    data = devm_kzalloc(&client->dev, sizeof(*data), GFP_KERNEL);
    data->client = client;
    i2c_set_clientdata(client, data);

    data->regmap = devm_regmap_init_i2c(client, &pmic_regmap_config);

    mutex_init(&data->lock);
    spin_lock_init(&data->status_lock);

    // Style B: Dedicated workqueue
    data->wq = alloc_workqueue("pmic-irq-wq",
                              WQ_HIGHPRI | WQ_MEM_RECLAIM,
                              1);    // max_active=1: serialize work items

    // Initialize work item
    INIT_WORK(&data->irq_work, pmic_irq_work_handler);

    // Register hardirq only
    ret = devm_request_irq(&client->dev,
                           client->irq,
                           pmic_hardirq,
                           IRQF_TRIGGER_LOW | IRQF_SHARED,
                           "pmic-irq", data);

    return 0;
}

static void pmic_remove(struct i2c_client *client)
{
    struct pmic_data *data = i2c_get_clientdata(client);
    destroy_workqueue(data->wq);    // flush and destroy
}
```

## Hardirq — Queues Work

```
static irqreturn_t pmic_hardirq(int irq, void *dev_id)
{
    struct pmic_data *data = dev_id;

    // For level-triggered: disable IRQ to prevent storm
    disable_irq_nosync(irq);
    // ^ Masks IRQ at GIC without waiting

    // Queue work to dedicated workqueue
    queue_work(data->wq, &data->irq_work);
    // ^ Adds work_struct into workqueue
    //   A kworker thread will pick it up

    return IRQ_HANDLED;
}
```

## Workqueue Handler (Bottom Half)

```
// BOTTOM HALF (process context, kworker thread)
// CAN sleep, CAN use mutex, CAN do full I2C transfers

static void pmic_irq_work_handler(struct work_struct *work)
{
    struct pmic_data *data = container_of(work,
                                           struct pmic_data,
```

```

                                irq_work);
unsigned int status, mask, irq_bits;

mutex_lock(&data->lock);    // CAN sleep here, process context

// Step 1: Read interrupt status registers (I2C)
regmap_read(data->regmap, PMIC_REG_INT_STATUS1, &status);
regmap_read(data->regmap, PMIC_REG_INT_MASK1, &mask);
irq_bits = status & ~mask;    // only unmasked interrupts

// Step 2: Handle each interrupt source
if (irq_bits & PMIC_INT_PWRON)
    input_report_key(data->input, KEY_POWER, 1);    // can sleep

if (irq_bits & PMIC_INT_ACOK)
    power_supply_changed(data->ac_psy);            // can sleep

if (irq_bits & PMIC_INT_TSHDN)
    thermal_zone_device_update(data->tz, ...);    // can sleep

// Step 3: Clear interrupt status (I2C write)
regmap_write(data->regmap, PMIC_REG_INT_STATUS1, irq_bits);

mutex_unlock(&data->lock);

// Step 4: Re-enable IRQ
enable_irq(data->client->irq);
// Now GIC will forward the interrupt again
}

```

## Workqueue Execution Flow Diagram

```

[HARDWARE]
    PMIC INT pin (level LOW) -> GIC SPI -> CPU exception

[HARDIRQ CONTEXT]
+-----+
| ell_irq -> gic_handle_irq -> handle_level_irq |
| -> handle_irq_event_percpu -> pmic_hardirq() |
|     disable_irq_nosync(irq)    <- mask at GIC |
|     queue_work(data->wq, &data->irq_work) |
|     +-- insert work_struct into workqueue |
|     +-- wake kworker thread if sleeping |
|     return IRQ_HANDLED |
+-----+
|         | hardirq returns quickly |
|         | (no masking needed, IRQ already disabled) |
|         v |
[PROCESS CONTEXT - kworker thread]
+-----+
| kworker/u8:2 (or pmic-irq-wq kworker) |
|     worker_thread() |
|     +-- process_one_work() |
|     +-- pmic_irq_work_handler(work) |
|         mutex_lock(&data->lock)    <- CAN sleep |
|         regmap_read(STATUS1)    <- I2C transfer |
|         +-- i2c_transfer() -> master_xfer() |
|         -> wait_for_completion() <- SLEEPS OK |
|         regmap_read(MASK1)    <- I2C transfer |
|         handle each interrupt source |
|         regmap_write(STATUS1)    <- I2C write |
|         mutex_unlock(&data->lock) |
|         enable_irq(irq)    <- re-enable GIC |
+-----+

```

## Delayed Work Variant

```

struct pmic_data {
    ...
    struct delayed_work irq_dwork;    // delayed work
}

```

```
};

// In probe:
INIT_DELAYED_WORK(&data->irq_dwork, pmic_irq_work_handler);

// In hardirq:
static irqreturn_t pmic_hardirq(int irq, void *dev_id)
{
    struct pmic_data *data = dev_id;
    // Schedule work to run after 50ms (debounce)
    queue_delayed_work(data->wq, &data->irq_dwork,
                       msecs_to_jiffies(50));
    return IRQ_HANDLED;
}
```

### 5.3 Comparison: All Three Bottom Half Mechanisms

| Property       | Threaded IRQ       | Tasklet          | Workqueue          |
|----------------|--------------------|------------------|--------------------|
| Context        | Dedicated kthread  | Softirq          | kworker thread     |
| Can sleep?     | YES                | NO               | YES                |
| Can use mutex? | YES                | NO               | YES                |
| Can do I2C?    | YES                | NO (usually)     | YES                |
| Scheduling     | SCHED_FIFO 50      | After hardirq    | Normal/HIGHPRI     |
| Latency        | Low (RT prio)      | Very low         | Medium             |
| Wake mechanism | IRQ_WAKE_THREAD    | tasklet_schedule | queue_work         |
| IRQ re-enable  | Automatic (kernel) | Automatic        | Manual: enable_irq |
| Real examples  | STK3310, ft5x06    | (avoid for I2C)  | MAX77686, da9063   |

### 5.4 Practical Decision Guide

```
Interrupt fires -> need bottom half?
|
|-- Need to do I2C/SPI/mutex/sleep?
|   |
|   |-- YES, latency-sensitive (PMIC, sensor)?
|   |   |-- Threaded IRQ (IRQF_ONESHOT)
|   |
|   |-- YES, complex multi-step work (USB, MMC)?
|   |   |-- Dedicated Workqueue (WQ_HIGHPRI)
|   |
|   |-- No sleep needed, pure data processing?
|       |-- Tasklet (but avoid I2C from tasklet!)
|           OR just do it in hardirq if truly minimal
```

## 6. Softirq vs Tasklet

### 6.1 What is a Softirq?

A softirq (software interrupt) is a statically defined, per-CPU, reentrant deferred execution mechanism. The kernel has a fixed, compile-time set of softirq vectors:

```
// include/linux/interrupt.h
enum {
```

```

HI_SOFTIRQ = 0,        // High-priority tasklets
TIMER_SOFTIRQ,         // Timer callbacks (hrtimer, etc.)
NET_TX_SOFTIRQ,        // Network transmit
NET_RX_SOFTIRQ,        // Network receive <- most active
BLOCK_SOFTIRQ,         // Block I/O completion
IRQ_POLL_SOFTIRQ,      // IRQ polling
TASKLET_SOFTIRQ,       // Normal tasklets <- tasklets run here
SCHED_SOFTIRQ,         // Scheduler (load balancing)
HRTIMER_SOFTIRQ,       // High-resolution timers
RCU_SOFTIRQ,           // RCU callbacks
NR_SOFTIRQS            // = 10
};

```

You cannot add new softirq vectors at runtime. Only core kernel subsystems (networking, block, timers) use raw softirqs directly.

## 6.2 What is a Tasklet?

A tasklet is a dynamically allocated, serialized wrapper built on top of softirqs — specifically `TASKLET_SOFTIRQ` (normal priority) and `HI_SOFTIRQ` (high priority).

```
Softirq (TASKLET_SOFTIRQ)
    +-- tasklet_action()
        +-- iterates per-CPU tasklet_vec list
        +-- calls your tasklet->func()
```

Every tasklet you create runs inside the `TASKLET_SOFTIRQ` handler. Tasklets are the driver-accessible interface to softirq execution.

### 6.3 Architecture Diagram

```
+-----+
| softirq_vec[] -- 10 fixed entries |
|                                     |
|   HI_SOFTIRQ      -> hi_tasklet_action() |
|   TIMER_SOFTIRQ   -> run_timer_softirq() |
|   NET_TX_SOFTIRQ  -> net_tx_action()     |
|   NET_RX_SOFTIRQ  -> net_rx_action()     |
|   BLOCK_SOFTIRQ   -> blk_done_softirq()  |
|   IRQ_POLL_SOFTIRQ-> irq_poll_softirq()  |
|   TASKLET_SOFTIRQ -> tasklet_action()    <-- drivers use this |
|   SCHED_SOFTIRQ   -> run_rebalance_domains() |
|   HRTIMER_SOFTIRQ -> hrtimer_run_softirq() |
|   RCU_SOFTIRQ     -> rcu_core_si()       |
+-----+
|                                     |
|                                     v TASKLET_SOFTIRQ fires
TASKLET LAYER (driver-accessible, dynamic)
+-----+
| Per-CPU tasklet_vec (linked list of pending tasklets) |
|                                                         |
| CPU0: tasklet_vec -> [ads_tasklet] -> [gpio_tasklet] -> NULL |
| CPU1: tasklet_vec -> [uart_tasklet] -> NULL                 |
| CPU2: tasklet_vec -> NULL                                   |
|                                                         |
| tasklet_action() iterates list, calls each tasklet->func() |
+-----+
```

## 6.4 Key Structural Differences

| Property        | Raw Softirq      | Tasklet    |
|-----------------|------------------|------------|
| Who can use it? | Core kernel only | Any driver |

| Property        | Raw Softirq                                                             | Tasklet                                                 |
|-----------------|-------------------------------------------------------------------------|---------------------------------------------------------|
| How many?       | Fixed: 10 (compile-time)                                                | Unlimited (dynamic alloc)                               |
| Reentrancy      | REENTRANT (SMP) -- same softirq can run on multiple CPUs simultaneously | SERIALIZED -- same tasklet runs on only ONE CPU at once |
| Locking needed? | YES -- must use per-CPU data or explicit locks                          | Less -- serialized by kernel                            |
| Can sleep?      | NO                                                                      | NO                                                      |
| Status          | Active, core use only                                                   | Deprecated (Linux 5.9+)                                 |
| Replacement     | N/A                                                                     | Threaded IRQ / Workqueue                                |

## 6.5 Reentrancy — The Critical Difference

### Raw Softirq: Reentrant Across CPUs

|                                                                                                                                                 |                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| CPU0<br>-----<br>NET_RX_SOFTIRQ fires<br>net_rx_action() runs<br>(same function, same time!)<br>Must use per-CPU data or<br>explicit spinlocks! | CPU1<br>-----<br>NET_RX_SOFTIRQ fires<br>net_rx_action() runs<br>(same function!) |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

The networking subsystem handles this using per-CPU receive queues (softnet\_data) -- no locking needed because each CPU processes its own queue.

### Tasklet: Serialized (Never Concurrent)

|                                                                                            |                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPU0<br>-----<br>ads_tasklet scheduled<br>TASKLET_STATE_RUN set<br>ads_tasklet_func() runs | CPU1<br>-----<br>ads_tasklet scheduled<br>tries to run ads_tasklet_func()<br>TASKLET_STATE_RUN already set!<br>-> reschedule for later<br>-> does NOT run concurrently |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The kernel uses TASKLET\_STATE\_RUN bit (atomic test-and-set) to guarantee only one CPU runs a given tasklet at a time.

## 6.6 Softirq Execution Points

| WHERE SOFTIRQS RUN                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------|
| 1. irq_exit() -- After every hardirq handler returns. Most common path for driver tasklets.                        |
| 2. local_bh_enable() -- When bottom-half processing is re-enabled (after spin_lock_bh / spin_unlock_bh).           |
| 3. ksoftirqd/N kernel thread -- When softirq load is too high (budget exceeded) OR raised outside hardirq context. |
| 4. do_softirq() called explicitly -- e.g., in networking: netif_rx_ni().                                           |

## 6.7 Softirq Budget and ksoftirqd

```
// kernel/softirq.c -- __do_softirq()
```

```

asmlinkage __visible void __softirq_entry __do_softirq(void)
{
    unsigned long end = jiffies + MAX_SOFTIRQ_TIME;    // 2ms budget
    int max_restart = MAX_SOFTIRQ_RESTART;            // 10 iterations

    pending = local_softirq_pending();

    restart:
    while (pending) {
        h = softirq_vec;
        while (pending) {
            if (pending & 1)
                h->action(h);    // call softirq handler
            h++;
            pending >>= 1;
        }

        pending = local_softirq_pending();
        if (pending) {
            if (time_before(jiffies, end) &&
                !need_resched() && --max_restart)
                goto restart;    // process more if budget allows

            // Budget exceeded -> wake ksoftirqd
            wakeup_softirqd();
        }
    }
}

```

## 6.8 Tasklet Internals — State Machine

Tasklet state bits (in tasklet\_struct.state):

```

TASKLET_STATE_SCHED (bit 0):
    Set by tasklet_schedule()
    Cleared by tasklet_action() when picked up for execution

TASKLET_STATE_RUN (bit 1):
    Set when tasklet->func() starts executing
    Cleared when tasklet->func() returns
    Prevents concurrent execution on multiple CPUs

```

State transitions:

```

[IDLE]
|
| tasklet_schedule()
v
[SCHED]  <- in per-CPU tasklet_vec list
|
| tasklet_action() picks it up
| test_and_clear_bit(SCHED)
v
[RUN]    <- func() executing on CPU N
|
| If tasklet_schedule() called during RUN:
|   -> sets SCHED bit again
|   -> re-added to list after func() returns
|
| func() returns
| clear_bit(RUN)
v
[IDLE]   (or back to SCHED if re-scheduled)

```

## 6.9 Locking Rules in Softirq/Tasklet Context

| Shared With...            | Lock to Use                                       |
|---------------------------|---------------------------------------------------|
| hardirq                   | spin_lock_irqsave() / spin_lock_irqrestore()      |
| process context           | spin_lock_bh() / spin_unlock_bh()                 |
| another CPU (raw softirq) | spin_lock() (needed for raw softirq, not tasklet) |
| same tasklet              | no lock needed (serialized by kernel)             |

## 6.10 Modern Kernel: Tasklets Are Deprecated

Starting from Linux 5.9+, tasklets are being deprecated in favor of threaded IRQs:

- ☐ Tasklets cannot sleep -> forces awkward non-blocking code
- ☐ Serialization guarantee makes them effectively single-threaded
- ☐ Threaded IRQs provide same serialization + can sleep
- ☐ Tasklets run at elevated priority (softirq) unnecessarily
- ☐ Hard to reason about locking (softirq vs process context)

Replacement: Old: tasklet\_init() + tasklet\_schedule(). New: request\_threaded\_irq() with IRQF\_ONESHOT OR workqueue with WQ\_HIGHPRI.

## 7. IRQ/Hardirq Stack

### 7.1 ARM32 (ARMv7) — Dedicated IRQ Mode Stack

On ARM32, the CPU has 7 processor modes, each with its own banked stack pointer (SP). When an IRQ fires, the hardware automatically switches to IRQ mode, which has its own dedicated SP\_irq.

ARM32 Processor Modes and their stacks:

| Mode    | SP register | Stack                                   |
|---------|-------------|-----------------------------------------|
| USR/SYS | SP_usr      | User/kernel task stack                  |
| SVC     | SP_svc      | Kernel supervisor stack (normal kernel) |
| IRQ     | SP_irq      | <- IRQ mode stack (hardirq runs here)   |
| FIQ     | SP_fiq      | FIQ mode stack                          |
| ABT     | SP_abt      | Abort mode stack                        |
| UND     | SP_und      | Undefined instruction stack             |

### ARM32 IRQ Stack Size and Usage

The IRQ mode stack on ARM32 is very small — typically 12 bytes (just enough to save a few registers before switching to SVC mode):

ARM32 IRQ Entry Stack Usage:

```
SP_irq (IRQ mode stack, ~12 bytes):
+-----+
| LR_irq (PC+4) | <- saved by hardware on IRQ entry
| SPSR_irq      | <- saved CPSR
+-----+
(immediately switches to SP_svc for real work)
```

```

SP_svc (SVC mode stack = task's kernel stack, 8KB):
+-----+
| pt_regs      | <- all registers saved here
| (r0-r15, cpsr) |
+-----+
| IRQ handler  |
| call frames  |
+-----+
| ...          |
+-----+
| task kernel  |
| stack (normal|
| execution)   |
+-----+

```

## 7.2 ARM64 (ARMv8/v9) — No Separate IRQ Mode Stack

ARM64 eliminated processor modes entirely. There is no "IRQ mode" with a banked SP. Instead, ARM64 uses Exception Levels (EL0/EL1/EL2/EL3) with a different SP model.

ARM64 has two SP choices at EL1: SP\_EL0 (SPSel=0, "thread stack pointer") and SP\_EL1 (SPSel=1, "handler stack pointer"). The Linux kernel runs with SPSel=1 at EL1, meaning SP\_EL1 is always active.

### ARM64 Exception Entry

Before IRQ:

```

SP = SP_EL1 -> points to current task's kernel stack
PC = somewhere in kernel code

```

Hardware on IRQ:

```

ELR_EL1 <- PC          (save return address)
SPSR_EL1 <- PSTATE      (save processor state)
PC <- VBAR_EL1 + 0x280  (jump to vector)
SP stays = SP_EL1      <- NO stack switch!

```

entry.S: kernel\_entry macro:

```

sub sp, sp, #S_FRAME_SIZE  <- grow SP_EL1 downward
stp x0, x1, [sp, #S_X0]    <- save registers onto SAME stack
...
// pt_regs now on the task's kernel stack

```

On ARM64, the hardirq uses the SAME kernel stack as the interrupted task (or the idle task's stack if interrupted during idle).

### ARM64 Kernel Stack During IRQ

```

SP_EL1 (task kernel stack, 16KB on ARM64):
+-----+ <- top of stack (high address)
| task_struct (thread_info) |
+-----+
| ...normal kernel execution |
| call frames                |
+-----+ <- SP before IRQ
| pt_regs (saved by          |
| kernel_entry macro)       | <- SP after kernel_entry
+-----+
| IRQ handler call frames   |
| gic_handle_irq()          |
| handle_domain_irq()       |
| handle_level_irq()        |
| handle_irq_event_percpu() |
| your_driver_isr()         |
+-----+ <- SP during ISR execution
| (remaining free stack)    |
+-----+ <- bottom of stack (low address)

```

## 7.3 IRQ Stack — CONFIG\_IRQ\_STACKS Option (ARM64)

Starting from Linux 4.9, ARM64 supports a dedicated per-CPU IRQ stack via CONFIG\_IRQ\_STACKS (enabled by default in most configs). IRQ\_STACK\_SIZE = THREAD\_SIZE = 16KB on ARM64.

### ARM64 with Dedicated IRQ Stack

| Task kernel stack (16KB): | Per-CPU IRQ stack (16KB): |
|---------------------------|---------------------------|
| +-----+                   | +-----+                   |
| task_struct               | (dedicated for IRQ)       |
| +-----+                   | +-----+                   |
| normal kernel             | gic_handle_irq()          |
| execution frames          | handle_domain_irq()       |
| +-----+                   | handle_level_irq()        |
| pt_regs saved here        | your_driver_isr()         |
| (kernel_entry)            |                           |
| +-----+                   | +-----+                   |
| ell_irq calls             |                           |
| call_on_irq_stack()       | (SP switches here)        |
| +-----+                   | +-----+                   |

```
// arch/arm64/kernel/entry.S (with CONFIG_IRQ_STACKS)
SYM_CODE_START_LOCAL(ell_irq)
    kernel_entry 1          // save pt_regs on task stack
    ...
    bl call_on_irq_stack    // switch to per-CPU IRQ stack
                           // then call handle_arch_irq
    ...
    kernel_exit 1          // restore from task stack
    eret
SYM_CODE_END(ell_irq)

// arch/arm64/kernel/irq.c
void call_on_irq_stack(struct pt_regs *regs,
                      void (*func)(struct pt_regs *))
{
    unsigned long stack = (unsigned long)raw_cpu_read(irq_stack_ptr);

    // Switch SP to IRQ stack, call func, switch back
    asm volatile(
        "mov    x19, sp\n"    // save current SP
        "mov    sp, %0\n"    // switch to IRQ stack
        "blr    %1\n"        // call handle_arch_irq
        "mov    sp, x19\n"    // restore SP
        : : "r"(stack), "r"(func) : "x19", "memory"
    );
}
```

### Why a Dedicated IRQ Stack?

Problem without dedicated IRQ stack:

```
-----
Task kernel stack = 8KB (ARM32) or 16KB (ARM64)
Normal kernel execution uses some of this
IRQ handler + its call chain uses more
Deep call chains -> stack overflow -> kernel panic
```

Especially dangerous with:

- Nested interrupts (IRQ during IRQ)
- Deep driver call stacks
- Large local variables in ISR

Solution with dedicated IRQ stack:

```
-----
IRQ always starts fresh on a known-clean 16KB stack
Task stack only holds pt_regs (fixed size)
No risk of overflowing task stack from IRQ
Each CPU has its own IRQ stack (no SMP contention)
```

## 7.4 Nested Interrupts and Stack Depth

Without nesting (Linux default -- IRQs masked during hardirq):  
 Normal code -> IRQ fires -> hardirq runs -> returns -> normal code  
 (IRQs masked, no nesting)  
 Stack depth: 1 level of IRQ frames

With nesting (if explicitly enabled, rare):  
 Normal code -> IRQ1 fires -> hardirq1 runs  
     -> IRQ2 fires (higher priority)  
     -> hardirq2 runs  
     -> returns  
     -> hardirq1 continues  
     -> returns -> normal code  
 Stack depth: 2 levels of IRQ frames  
 (Linux generally avoids this -- uses GIC priority masking instead)

## 7.5 Stack Overflow Detection

```
// CONFIG_SCHED_STACK_END_CHECK
// Places a magic value at the bottom of each stack
// Checked on every context switch and IRQ entry

#define STACK_END_MAGIC 0x57AC6E9F

// arch/arm64/kernel/entry.S
// kernel_entry macro checks stack canary:
ldr    x25, [tsk, #TSK_STACK_CANARY]
// if corrupted -> panic("stack-protector: Kernel stack is corrupted")
```

Stack layout with overflow detection:

```
+-----+ <- top (high addr)
| task_struct / thread_info |
+-----+
| usable stack space       |
| (grows downward)        |
|                           |
| v v v v v v v v v v v   |
|                           |
+-----+
| STACK_END_MAGIC (canary) | <- if overwritten -> panic
+-----+ <- bottom (low addr)
```

## 7.6 Checking Stack Usage in Practice

### From T32 (Lauterbach JTAG)

```
// Check current SP during IRQ
r.s SP

// Check IRQ stack base and current usage
v.v irq_stack      // per-CPU IRQ stack array
v.v irq_stack_ptr   // current IRQ stack pointer

// Dump call stack during IRQ
bt                // backtrace
v.v %t %l task_struct // check current task context
v.v irq_desc.action // check irqaction chain
```

### From Kernel (in driver)

```
void my_isr(int irq, void *dev_id)
{
```

```

// Check how much stack is left
unsigned long sp = current_stack_pointer;
unsigned long stack_base = (unsigned long)task_stack_page(current);
pr_debug("ISR stack remaining: %lu bytes\n",
        sp - stack_base);
}

```

## 7.7 ARM32 vs ARM64 IRQ Stack Comparison

| Property             | ARM32 (ARMv7)                              | ARM64 (ARMv8/v9)                           |
|----------------------|--------------------------------------------|--------------------------------------------|
| IRQ mode stack?      | YES -- SP_irq (tiny, ~12 bytes, transient) | NO -- no IRQ mode                          |
| Real work stack      | SP_svc (task kernel stack, 8KB)            | SP_EL1 (task kernel stack, 16KB)           |
| Dedicated IRQ stack? | NO (uses task stack)                       | YES with CONFIG_IRQ_STACKS (16KB)          |
| Stack switch?        | IRQ->SVC mode switch (hardware)            | call_on_irq_stack() (software, if enabled) |
| pt_regs saved on     | SVC stack                                  | Task stack (always)                        |
| Stack size           | 8KB (THREAD_SIZE)                          | 16KB (THREAD_SIZE)                         |
| Overflow risk        | Higher (shared with task)                  | Lower (dedicated IRQ stack separates them) |

Key takeaway: The hardirq/ISR always has a stack. On ARM32 it briefly uses the IRQ mode stack then switches to the SVC (task kernel) stack. On ARM64 with CONFIG\_IRQ\_STACKS, it uses a dedicated per-CPU IRQ stack for the handler body, while pt\_regs is always saved on the task's kernel stack. This is why you must never use large local arrays in ISRs — stack space is limited and shared.

## 8. Interview Summary — Quick Reference

### 8.1 ARM Interrupt Handling — Quick Recall

Why: Async hardware event notification without polling; provides priority management + SMP routing.

#### GIC (Generic Interrupt Controller)

- ☐ GICv2: Distributor (GICD) + CPU Interface (GICC) — MMIO based
- ☐ GICv3: GICD + Redistributor (GICR) + ITS (for MSI/PCIe) — system register based
- ☐ Interrupt types: SGI (0–15, IPI), PPI (16–31, per-CPU timers), SPI (32–1019, peripherals), LPI (8192+, MSI)

#### ARM64 Exception Entry

- ☐ No modes — uses Exception Levels (EL0–EL3)
- ☐ IRQ vector at VBAR\_EL1 + 0x280 (kernel) or +0x480 (userspace)
- ☐ Hardware saves PC→ELR\_EL1, PSTATE→SPSR\_EL1, jumps to vector

## IRQ Domains

- ❑ Maps hwirq → virq (Linux virtual IRQ number)
- ❑ Linear revmap for dense spaces (GIC SPIs), radix tree for sparse (LPIs)
- ❑ Hierarchical: GIC → GPIO → device (chained domains)

## 8.2 SPI Interrupt Flow — 10-Step Summary

| Step | What Happens                                                      |
|------|-------------------------------------------------------------------|
| 1    | Peripheral asserts SPI line to GIC                                |
| 2    | GICD checks enabled/priority/target -> forwards to CPU            |
| 3    | CPU takes exception -> VBAR_EL1 + 0x280                           |
| 4    | kernel_entry: saves pt_regs on stack                              |
| 5    | gic_handle_irq(): reads ICC_IAR1_EL1 (ACK + get hwirq)            |
| 6    | irq_find_mapping(domain, hwirq) -> virq                           |
| 7    | irq_desc[virq].handle_irq -> handle_level_irq                     |
| 8    | handle_irq_event_percpu() -> calls ALL irqaction handlers         |
| 9    | irq_chip->irq_eoi() -> ICC_EOIR1_EL1 (priority drop + deactivate) |
| 10   | kernel_exit + eret -> resume interrupted code                     |

Shared IRQs (IRQF\_SHARED): All handlers called sequentially. Each driver checks own hardware status register. Returns IRQ\_NONE (not mine) or IRQ\_HANDLED. dev\_id is unique key — identifies handler on free\_irq().

## 8.3 Threaded IRQs — Key Points

- ❑ Split: hardirq (top half, fast) + kthread (bottom half, can sleep)
- ❑ IRQ\_WAKE\_THREAD: hardirq return value to wake thread
- ❑ IRQF\_ONESHOT: keeps IRQ masked until thread completes — mandatory for level-triggered
- ❑ Thread name: irq/<virq>-<name>, runs at SCHED\_FIFO priority 50
- ❑ Use when: I2C/SPI bus access, mutex, blocking operations needed

*One-liner: "Threaded IRQs let you split interrupt handling — minimal work in hardirq context (timestamp, ACK), then heavy work in a dedicated kthread that can sleep and acquire mutexes."*

## 8.4 I2C Subsystem — Key Structures

| Structure     | Role                                           |
|---------------|------------------------------------------------|
| i2c_adapter   | One physical I2C bus (controller hardware)     |
| i2c_algorithm | Hardware transfer ops (master_xfer)            |
| i2c_client    | One device on the bus (addr, irq, adapter ptr) |
| i2c_driver    | Driver matching i2c_client (probe/remove)      |
| i2c_msg       | Single transfer unit (addr, flags, len, buf)   |

Transfer chain: regmap\_read() -> i2c\_smbus\_read\_byte\_data() -> i2c\_transfer() -> adap->algo->master\_xfer() -> hardware

STK3310 key design: Uses request\_threaded\_irq() with IRQF\_ONESHOT because reading interrupt status requires I2C (sleeping) in the thread handler.

## 8.5 Bottom Half Comparison

| Mechanism  | Threaded IRQ      | Tasklet        | Workqueue      |
|------------|-------------------|----------------|----------------|
| Context    | Dedicated kthread | Softirq        | kworker thread |
| Can Sleep? | YES               | NO             | YES            |
| I2C OK?    | YES               | NO             | YES            |
| Latency    | Low (RT)          | Very low       | Medium         |
| Use case   | I2C/SPI sensors   | Fast non-sleep | PMIC, USB, MMC |

Decision rule: Need I2C/mutex? -> Threaded IRQ or Workqueue. Never tasklet.

## 8.6 Softirq vs Tasklet — Key Differentiator

- ☐ Softirq: Fixed 10 vectors, reentrant (same softirq runs on multiple CPUs simultaneously), core kernel only
- ☐ Tasklet: Dynamic, serialized (same tasklet instance never concurrent), built on TASKLET\_SOFTIRQ
- ☐ Tasklets deprecated since Linux 5.9 — use threaded IRQs instead

*One-liner: "Softirqs are reentrant per-CPU mechanisms for core subsystems like networking. Tasklets are serialized wrappers on top of softirqs for drivers, but deprecated — we use threaded IRQs now."*

## 8.7 IRQ Stack — Quick Facts

- ☐ ARM32: IRQ mode stack is tiny (~12 bytes), real work on SVC (task) stack (8KB)
- ☐ ARM64: No IRQ mode; with CONFIG\_IRQ\_STACKS, uses dedicated per-CPU 16KB IRQ stack via call\_on\_irq\_stack()
- ☐ Why: Prevents IRQ handler call chain from overflowing the task's kernel stack
- ☐ pt\_regs always saved on task stack; handler runs on IRQ stack

## 8.8 Quick Interview Q&A

| Question                                                          | Answer                                                                                                                                                    |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| How does kernel know which driver to call for a shared interrupt? | It calls ALL handlers. Each driver checks its own hardware status register and returns IRQ_NONE or IRQ_HANDLED.                                           |
| Why IRQF_ONESHOT?                                                 | For level-triggered interrupts with threaded handlers — keeps the line masked until the thread clears the hardware source, preventing an interrupt storm. |
| Difference: IRQ domain vs irq_chip?                               | IRQ domain maps hwirq->virq (numbering). irq_chip abstracts the controller hardware ops (mask, unmask, EOI, set_affinity).                                |

| Question                                                             | Answer                                                                                                                                                                          |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Can you sleep in a hardirq?                                          | No. Hardirq runs in interrupt context with preemption disabled. Use threaded IRQ or workqueue for sleeping.                                                                     |
| What happens if you call <code>i2c_transfer()</code> from a tasklet? | "BUG: scheduling while atomic" — because <code>i2c_transfer()</code> acquires a mutex internally, which can sleep. Tasklets run in softirq context where sleeping is illegal.   |
| How does GICv3 differ from GICv2?                                    | System registers instead of MMIO for CPU interface, Redistributor per-CPU for SGI/PPI, ITS for MSI/LPI translation, supports >8 CPUs, affinity routing via GICD_IROUTER.        |
| What does reading <code>ICC_IAR1_EL1</code> do?                      | Three things simultaneously: (1) ACKs the interrupt, (2) Returns the hwirq number, (3) Deactivates the interrupt in GIC.                                                        |
| What is <code>irq_exit()</code> responsible for?                     | Decrements hardirq preempt count and checks if any softirqs are pending. If not in interrupt context and softirqs are pending, calls <code>invoke_softirq()</code> to run them. |

## 8.9 Key Kernel Files Reference

| File                                      | Purpose                                                                                                       |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>arch/arm64/kernel/entry.S</code>    | Exception vectors, <code>el1_irq</code> , <code>kernel_entry</code> /exit macros                              |
| <code>drivers/irqchip/irq-gic-v3.c</code> | GICv3 driver, <code>gic_handle_irq</code> , domain ops                                                        |
| <code>kernel/irq/irqdomain.c</code>       | IRQ domain core: mapping, creation, lookup                                                                    |
| <code>kernel/irq/chip.c</code>            | <code>irq_chip</code> helpers, <code>handle_edge_irq</code> , <code>handle_level_irq</code>                   |
| <code>kernel/irq/manage.c</code>          | <code>request_irq</code> , <code>request_threaded_irq</code> , <code>free_irq</code>                          |
| <code>kernel/irq/handle.c</code>          | <code>handle_irq_event_percpu</code> , <code>__irq_wake_thread</code>                                         |
| <code>kernel/softirq.c</code>             | <code>__do_softirq</code> , <code>raise_softirq</code> , <code>tasklet_action</code> , <code>ksoftirqd</code> |
| <code>kernel/workqueue.c</code>           | <code>alloc_workqueue</code> , <code>queue_work</code> , <code>worker_thread</code>                           |
| <code>drivers/iio/light/stk3310.c</code>  | STK3310 IIO driver (I2C + threaded IRQ example)                                                               |
| <code>drivers/i2c/i2c-core-base.c</code>  | I2C core: <code>i2c_transfer</code> , <code>i2c_new_client_device</code>                                      |

— End of Document — Prepared for: Sandeep Kumar